Chapter 1. Introduction and Background

1.1 Introduction

Large language models (LLMs) have played a pivotal part in the advancement of artificial intelligence, driving innovations across a spectrum of applications including but not limited to automated content creation [1–6], intricate question-answering systems [7–9], and society simulation [10–12]. Despite their transformative impact, the transparency of the inner mechanisms of LLMs remains a significant barrier, limiting both their development and broader application.

The challenge of interpreting and understanding the rationale behind the decisions made by LLMs has garnered significant attention within the research community [13–17].

Current mainstream LLMs utilize the transformer architecture which contains many layers of self-attending neurons to capture the complexities of natural language. The activation patterns of individual neurons within the transformer architecture play a critical role in determining how LLMs process these complexities and infer subsequent tokens [18]. While processing different text samples that are diverse in subject matter, tone, purpose, etc., a behavioral pattern of activation emerges for each neuron. Analyzing these activation patterns offers valuable insights into the inner workings of LLMs, enhancing interpretability and paving the way for greater transparency and trust in their applications [19–21].

The complexity of interpreting and visualizing LLMs stems from their massive scale and the intricate interactions between their components. Modern LLMs typically contain hundreds of billions of parameters distributed across numerous layers, making it challenging to trace how specific inputs lead to particular outputs. Traditional approaches to model interpretation have primarily focused on analyzing attention patterns and token-level interactions [19–21]. While these methods have provided valuable insights, they often fall short of explaining the specific roles of individual neurons in the model's decision-making process. Furthermore, existing visualization approaches are often concerned with visualizing the entire model at a high level [**meta**, 22, 23].

Recent research has highlighted the importance of understanding individual neuron behavior in transformer models [14]. Such studies have shown that specific neurons can specialize in detecting particular linguistic features, semantic concepts, or even factual knowledge. However, identifying and analyzing these specialized neurons remains a labor-intensive process, often requiring researchers to manually write scripts for each hypothesis they want to test.

The growing need for model interpretability is driven not only by technical considerations but also by practical concerns about model reliability and ethical deployment. As LLMs become increasingly integrated into critical applications across a wide range of domains, understanding their decision-making processes becomes essential for ensuring safe and responsible AI development. This understanding can help identify potential failure modes, mitigate biases, and improve model performance through targeted interventions.

We believe there is a gap that exists for an exploratory visual analysis tool that can aid in these preliminary investigations, eliminating the need for bespoke scripts to test early research hypotheses. To better understand the requirements for investigating neuron behavior, we conducted a formative study with LLM researchers. Through these discussions we identified several key problems that concern LLM researchers, such as *"Which neurons were most influential for this prompt?"*, *"What are the specific roles of these neurons?"*, etc.

To effectively address these concerns, we designed NeuronautLLM, a visual analysis system that empowers efficient neuron exploration and allow users to intuitively understand the model behavior triggered by their prompt inputs. NeuronautLLM features three key views: (1) *Token effect and prediction view* which allows users to observe how individual sections of the input prompt influenced the model's predictions as well as presenting a visual breakdown of the model's final predicted output. (2) *Neuron space view* employs a novel projection technique, positioning influential neurons based on both their activations and semantic explanations concurrently, thereby facilitating the exploration of neurons in a semantic space. (3) *Neuron explanation view* offers in-depth insights into each neurons function within the model. Finally, to evaluate our approach, we devised two case studies that cover two distinct areas of LLM research - model reasoning and social bias - to showcase NeuronautLLM's versatility in performing practical model interpretation. NeuronautLLM was also evaluated by five domain

experts who validated its effectiveness as a tool for LLM research.

The key contributions of our paper are:

1. A formative study investigating the requirements for analyzing neuron activation in LLMs
2. NeuronautLLM, a visual analysis system featuring an effective neuron projection method and novel visual designs
3. Open-sourced code for both the NeuronautLLM application [1] and the code written to collate the application's static data [2]
4. Two case studies along with five expert interviews including a usability study to evaluate the efficacy of our approach and its real world application

1.2 Related work

1.2.1 Neural network visualization

Using visualizations to communicate the structure and interpretability of deep neural networks (DNN) has been an area of active research for quite some time [24]. By visualizing a network's structure and even its internal algorithms, the model architecture can be made easier to understand.

Model architecture visualization Early work in DNN visualization focused on presenting representations of model structures. In 2015, Harley [25] developed an interactive visualization that allowed users to input handwritten digits and observe the resulting activation patterns within a convolutional neural network (CNN). Wang et al. [26] presented CNN Explainer, a web-based tool aimed at helping beginners understand CNNs through interactive model overviews and dynamic visualizations. More recent approaches have explored novel ways to visualize DNNs architectures. Rogawski [27] introduced a 3D visualization technique that leverages established methods from neural network optimization to estimate the importance of different model components, resulting in a 3D model representation of a neural network.

[1] https://github.com/ollierwoodman/NeuronautLLM

[2] https://github.com/ollierwoodman/NeuronautLLM-database

Model state visualization With an increase in the complexity of DNNs, particularly transformer-based models, researchers began to develop tools for visualizing their internal states. Vig [22] introduced an open-source tool for visualizing the multi-layer, multi-head attention mechanism in transformers, allowing for model interpretation and the detection of potential biases. Hohman et al. [28] presented Summit, an interactive system that summarizes and visualizes the features learned by a deep learning model and how they interact to make predictions. More recently, tools have emerged that specifically target the visualization of transformer models. Gao et al. [23] developed TransforLearn, the first interactive visual tutorial designed to help beginners understand transformers through architecture-driven and task-driven exploration. Tufanov et al. [29] introduced the LM Transparency Tool (LM-TT), an interactive toolkit for analyzing the entire prediction process of transformer-based language models, from top-layer representations to fine-grained model components.

From a visualization perspective, our work builds upon existing research by forming an approach to visualizing neurons spatially based on their activation patterns, each neuron is then depicted using metrics that measure its influence. With this approach, we aim to enable easier exploration and identification of neurons that are influential for a user-defined prompt.

1.2.2 LLM interpretability

The interpretability of components in neural networks, particularly within the domain of natural language processing and LLMs, has been a subject of ongoing debate and investigation.

Attention as explanation Early studies, such as those by Li et al. [30], suggested that attention mechanisms could provide valuable insights into the internal workings of neural models, with attention weights being interpreted as indicators of feature importance. This perspective was further explored in works like Xu et al. [31] and Choi et al. [32], where attention mechanisms were leveraged to explain model predictions and highlight relevant input features.

Challenging the interpretability of attention However, Jain & Wallace [33] questioned the validity of this interpretation approach. They conducted extensive experiments across various NLP tasks and found that attention weights often did not correlate with established measures of feature importance, such as gradients. Moreover, they demonstrated the possibility of con-

structing alternative attention distributions that yielded similar predictions despite attending to different input features, thus challenging the notion that attention weights could be reliably used to explain model behavior.

Reconciling contrasting viewpoints Subsequent research, like Vashishth et al. [34], sought to reconcile the seemingly contradictory findings regarding attention interpretability. Vashishth et al. argued that the interpretability of attention weights depends on the specific task and model architecture. They showed that while attention might not be a reliable indicator of feature importance in text classification tasks, it plays a crucial role in tasks like natural language inference and neural machine translation. They proposed that attention is interpretable when it is computed using two vectors that are both functions of the input, and not interpretable when the input has only a single sequence.

Scaling interpretability to LLMs Recent work has focused on extending interpretability research to LLMs, which present unique challenges due to their scale and complexity. Foote et al. [35] proposed Neuron to Graph (N2G), a tool that automatically converts neuron activations in LLMs into interpretable graphs, aiming to streamline the process of neuron interpretation by providing a visual representation of neuron behavior and enabling the quantitative assessment of the graph's accuracy in capturing neuron activations.

As laid out by Foote et al. in their suggestions for directions of future work, Bills et al. [13] introduced an automated approach to explain neuron activations in LLMs with a wealth of text samples. Their method involves iteratively generating natural language explanations for neuron activations, simulating activations based on these explanations, and scoring the explanations based on the agreement between simulated and real activations. This approach allows for the quantitative measurement of interpretability and provides insights into the functionality of individual neurons.

We intend to visualize neurons in a space that is more meaningful from a function and semantic perspective rather than visualizing them according to their place in the architecture of the model. This approach will make the structure of the model more opaque, but we believe that accurately representing transformer architecture is not strictly necessary for model interpretation and simply places additional constraints on visualization that do not serve the primary purpose.

Our approach aims to visualize the ‘influence’ of each neuron, thereby making the contribution of each neuron towards a user-defined expected out interpretable. We also intend to present details about each neuron so that users can investigate patterns between a neuron’s influence upon the model output and the neuron’s function.

Chapter 2. Foundational Technologies

2.1 Transformer-based language models

The transformer architecture is a type of deep neural network model that has proven to be highly effective in natural language processing (NLP) tasks [18, 36]. Unlike traditional recurrent neural networks that process sequential data in a linear fashion, transformers leverage a 'self-attention' mechanism as laid out by Vaswani et al. [18] and depicted in Figure 2.1. This mechanism allows the model to weigh the importance of different words in a sequence relative to one another, enabling it to capture deep dependencies and contextual relationships within natural language.

The transformer accepts natural language as input. This input is firstly split into a chain of *tokens* (whole words or sub-words). The tokens are then transformed into an embedding which serves as a numerical representation of each token. Each embedding captures the meaning and relationships between words.

In the transformer architecture, the *Multilayer perceptron* (MLP) neuron serves as a crucial component within the feed-forward network of each transformer block. Its primary purpose is to introduce non-linearity and complexity to the model. The MLP neuron consists of two linear layers with an activation function in between, creating the self-attention mechanism. This structure allows it to learn complex patterns and relationships within the input data, enhancing the model's ability to capture intricate contextual information and dependencies. Hence, the MLP neurons and their activations in language models serve as a key subject of interpretability research.

Finally, after processing all of the context provided in the input, the transformer then outputs a number known as a *logit* for every token it can possibly output. Using a *softmax* function, the logits are transformed into probabilities that can tell us the chance of each token being chosen as the next one in the sequence.

By continuously querying the transformer model, we can make the transformer generate

sentences, paragraphs or even pages of natural language.

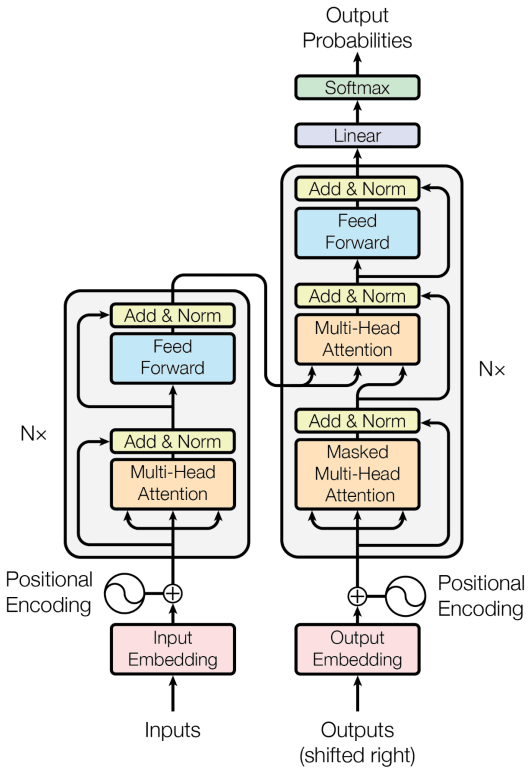

Figure 2.1 A diagram detailing the transformer architecture from the landmark paper by Vaswani et al. [18].

2.2 Transformer Debugger

As artificial neural networks grow increasingly complex and ubiquitous in various applications, understanding their internal mechanisms has become a critical challenge in the field of artificial intelligence. Transformer Debugger (TDB), developed by OpenAI's Superalignment team, represents a significant advancement in model interpretability tools, specifically designed to investigate the behavioral patterns of small language models through a combination of automated interpretability techniques and sparse autoencoders [37].

TDB's architecture facilitates an innovative approach to model analysis by enabling researchers to conduct rapid exploratory investigations without immediate code implementation. This tool's distinctive feature lies in its ability to perform forward pass interventions while si-

multaneously observing their effects on specific behavioral outcomes. The system operates by identifying and analyzing key components, including neurons, attention heads, and autoencoder latents, that contribute to particular model behaviors. Furthermore, it generates automated explanations for component activation patterns and maps the interconnections between these elements to uncover underlying neural circuits.

The tool's infrastructure is comprised three primary components: a React-based neuron viewer application that serves as the main interface, an activation server backend that manages model inference and data delivery, and a specialized inference library focusing on GPT-2 models and their associated autoencoders. This comprehensive framework allows researchers to address fundamental questions about model behavior, such as token selection preferences and attention mechanisms, thereby advancing our understanding of neural network decision-making processes.

TDB contributes to the field of AI interpretability by offering researchers an advanced platform for investigating the intricate behaviors of language models through a user-friendly interface while maintaining robust analytical capabilities. This tool demonstrates the growing emphasis on developing systematic approaches to understanding and debugging artificial neural networks, particularly in the context of language processing systems.

2.3 Explaining neuron behaviour

Bills et al. present a novel methodology for automatically explaining and validating the behavior of individual neurons within large language models using GPT-4 [13]. The researchers developed a three-step approach: first, using GPT-4 to generate explanations of neuron behavior by analyzing activation patterns; second, employing GPT-4 to simulate neuron activations based on these explanations; and third, quantitatively scoring the explanations by comparing simulated activations against actual neuron behavior.

The researchers conducted extensive experiments to validate and improve their approach, including comparing against baseline methods like token-based prediction and implementing an explanation revision process that improved scores through iterative refinement. Their anal-

ysis revealed several important trends, including that explanation quality generally decreases in deeper layers of larger models and that using more sparse activation functions can improve neuron interpretability while moderately impacting model performance. The team also developed ”neuron puzzles” - synthetic neurons with known ground truth explanations - to evaluate their methodology’s effectiveness.

The study has made significant contributions to the field of AI interpretability by releasing several valuable resources to the research community. These include a comprehensive dataset of explanations for all neurons in some models of the GPT-2 family, code for implementing their explanation and scoring methodology, and an interactive neuron visualization interface. This public release enables other researchers to explore neuron behavior, validate findings, and build upon their work in developing more interpretable AI systems.

2.4 UMAP

Dimensionality reduction techniques play a crucial role in modern data analysis and machine learning applications, particularly when dealing with high-dimensional datasets like those present in machine learning and large language model domains. This high dimensionality presents significant computational and interpretative challenges. Among these techniques, UMAP (Uniform Manifold Approximation and Projection) emerges as a sophisticated manifold learning algorithm that effectively addresses the limitations of existing approaches while offering superior performance characteristics [38].

UMAP’s theoretical foundation is deeply rooted in advanced mathematical concepts, specifically Riemannian geometry and algebraic topology. This robust mathematical framework enables UMAP to capture both local and global topological structures within high-dimensional data spaces, facilitating more accurate lower-dimensional representations. The algorithm’s construction reflects a careful balance between theoretical rigor and practical applicability, resulting in an implementation that scales efficiently to real-world datasets.

In comparative analyses, UMAP demonstrates performance metrics that rival or exceed those of t-SNE, particularly in terms of visualization quality. A distinguishing feature of UMAP

is its enhanced preservation of global data structure, which provides researchers with more faithful representations of their high-dimensional datasets. Additionally, UMAP exhibits superior computational efficiency, reducing processing time requirements while maintaining high-quality results.

A significant advantage of UMAP lies in its flexibility regarding embedding dimensionality. Unlike some alternative techniques that impose restrictions on the target dimension, UMAP can project data into arbitrary-dimensional spaces. This versatility positions UMAP as a valuable tool not only for visualization purposes but also as a general-purpose dimensionality reduction technique in broader machine learning applications.

2.5 Sentence embedding

Sentence embedding is a process wherein semantic content is transformed into dense vector representations within a high-dimensional space. These vector-based representations are generated through complex neural architectures that have been trained to encode lexical, syntactic, and semantic relationships into fixed-dimensional numerical sequences, typically comprising hundreds or thousands of dimensions. This mathematical transformation enables the conversion of linguistic meaning into a computationally tractable format while preserving the nuanced semantic relationships inherent in natural language.

The fundamental utility of sentence embeddings derives from their capacity to capture semantic similarity through geometric relationships in vector space. When two sentences convey analogous semantic content, their corresponding embedding vectors exhibit proximity in the embedding space, independent of surface-level lexical overlap. This property enables the quantification of semantic similarity through vector operations such as cosine similarity or Euclidean distance, providing a mathematically rigorous framework for comparing textual meaning that transcends traditional keyword-based approaches.

These vector-based representations have become instrumental in numerous natural language processing applications and computational linguistics research. In information retrieval contexts, sentence embeddings facilitate semantic search functionality by enabling meaning-

based document retrieval rather than relying solely on lexical matching. They also serve as foundational features for various downstream tasks including document clustering, semantic similarity assessment, and duplicate detection. Moreover, sentence embeddings have proven crucial in advanced natural language understanding systems, including question-answering frameworks and cross-lingual applications, where they contribute to the underlying representational architecture of contemporary transformer-based language models.

2.6 BERTopic

BERTopic is an algorithm that identifies latent topics in collections of text data [39]. It represents a significant advancement in neural topic modeling, introducing an innovative approach that combines the power of transformer-based language models with clustering techniques and a novel class-based TF-IDF procedure. This methodology addresses fundamental limitations of traditional topic modeling approaches, particularly their reliance on bag-of-words representations which fail to capture semantic relationships between words. By leveraging contextual embeddings from pre-trained language models, BERTopic enables more nuanced and semantically meaningful topic extraction from document collections.

The algorithm operates through a three-stage process that distinguishes it from conventional topic modeling approaches. Initially, it generates document embeddings using pre-trained transformer models like SBERT (Sentence-BERT) [40], capturing rich contextual information. These high-dimensional embeddings undergo dimensionality reduction through UMAP before being clustered using HDBSCAN [38, 41], which effectively identifies document groups of varying densities. The final stage involves a innovative class-based variation of TF-IDF that generates topic representations from these clusters, moving beyond the traditional centroid-based approaches used in other embedding-based topic models. This methodology allows BERTopic to identify more coherent and diverse topics while maintaining computational efficiency.

BERTopic demonstrates particular utility in several key applications, including dynamic topic modeling and the analysis of large-scale document collections. Its flexibility in language model selection and preprocessing approaches makes it adaptable to various use cases and com-

putational environments, as highlighted in Figure. 2.2. The model maintains competitive performance metrics in terms of topic coherence and diversity across different datasets, though it does face certain limitations, such as its assumption that each document contains only a single topic and its reliance on bag-of-words for final topic representation despite using contextual embeddings for clustering.

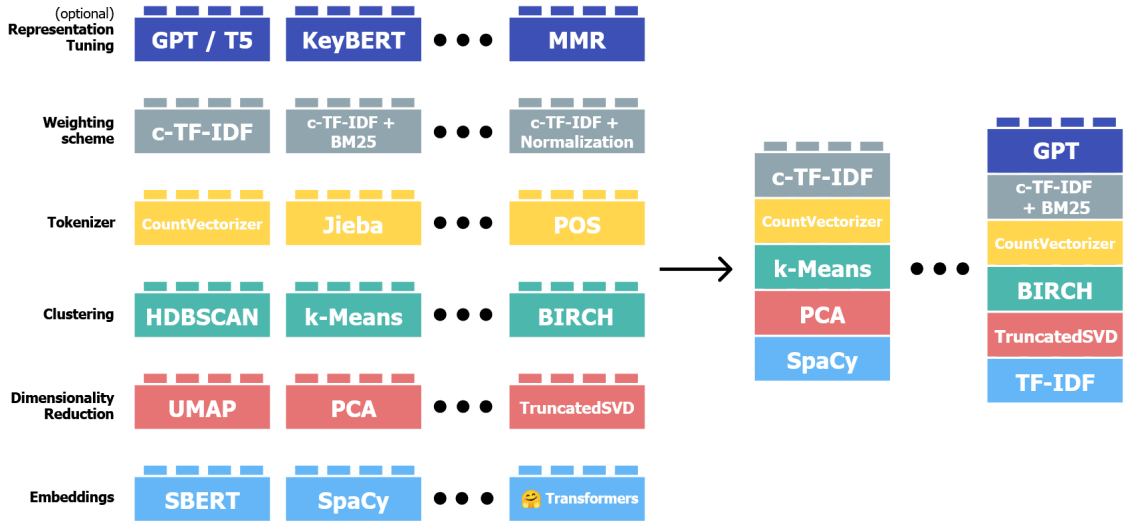

Figure 2.2 A diagram highlighting the modularity of the BERTopic algorithm. Image source: `https://maartengr.github.io/BERTopic/algorithm/algorithm.html`

2.7 Development tools and frameworks

React

React[1] is an open source declarative JavaScript library maintained by Meta and the open source community. It represents a major shift in front-end architectural methodologies through its implementation of a virtual Document Object Model (DOM) and component-based development paradigm. The framework's core architecture facilitates the construction of interactive user interfaces through a unidirectional data flow model, wherein components, fundamental building blocks of React applications, maintain internal state management capabilities while implementing a reconciliation algorithm that optimizes DOM manipulation operations through differential analysis of virtual DOM representations. This architectural approach, coupled with Re-

[1] `https://react.dev/`

act's robust ecosystem of developer tools and extensive community-maintained component libraries, enables the development of scalable, maintainable web applications while significantly reducing the complexity traditionally associated with managing dynamic user interface states and component lifecycle events. React's implementation of the Virtual DOM and component-based architecture has profound implications for application performance optimization and development workflow efficiency, establishing it as a foundational technology in modern web development methodologies.

Next.js

Next.js[2] is an open source, powerful web framework based on React and developed by Vercel, Inc. It streamlines the development of modern web applications by providing an intuitive architecture and essential features out of the box. At its core, Next.js extends React's capabilities by adding server-side rendering, automatic code splitting, and simplified routing, which means developers can create fast, SEO-friendly applications without manually configuring complex tooling. The framework's built-in features handle many common development challenges - for instance, it automatically optimizes images, manages API routes, and supports both static and dynamic content generation, making it easier to build applications that scale.

Tailwind CSS

Tailwind CSS[3] is a utility-first, open source CSS framework developed by Tailwind Labs. Instead of creating custom class names and writing corresponding CSS rules, it allows developers to use predefined utility classes to style HTML elements. This approach allows for rapid development by avoiding context-switching between CSS and other files, while still maintaining consistency through a predefined design system of colors, spacing, typography, and other visual elements. The framework automatically removes unused CSS utility classes in production, resulting in highly optimized stylesheets.

shadcn

shadcn/ui[4] is a vast collection of reusable components built upon the foundation of Radix

[2] https://nextjs.org/

[3] https://tailwindcss.com/

[4] https://ui.shadcn.com/

UI[5] primitives and styled using Tailwind CSS. It represents a deliberate departure from traditional component libraries by implementing a unique 'copy-paste' architecture, wherein developers selectively install and incorporate only the specific components they require into their codebase. This approach affords developers complete control over their component implementation while maintaining full access to the source code. The library adheres to established accessibility standards through its use of Radix UI primitives, ensuring that complex interactive elements like dropdowns, modals, and form controls maintain proper ARIA attributes and keyboard navigation support. Components are styled using Tailwind CSS's utility classes and CSS variables, enabling straightforward customization while preserving a cohesive design system. The library's architecture emphasizes developer autonomy and customization flexibility, making it particularly well-suited for projects that require a high degree of control over component implementation while still benefiting from pre-built, accessible functionality.

D3.js

D3.js (Data-Driven Documents)[6] is an open source JavaScript library engineered specifically for data visualization and manipulation within web browsers. The library implements a declarative programming paradigm wherein developers bind data directly to Document Object Model (DOM) elements, facilitating the creation of dynamic, interactive visualizations through SVG, Canvas, and HTML elements.

The library's architectural foundation rests upon its data join concept, which establishes a programmatic relationship between abstract data and concrete visual elements. This paradigm enables developers to define how data should be represented visually through a series of transformations and transitions, while D3 efficiently manages the underlying DOM manipulations and rendering processes. The library provides comprehensive control over the final visual representation through a suite of mathematical and statistical functions, scales, and coordinate system transformations.

[5] https://www.radix-ui.com/

[6] https://d3js.org/

Chapter 3. Requirement Analysis and System Design

3.1 Formative Study

This section introduces our interview study, its findings, and the design goals of NeuronautLLM informed by interviews.

3.1.1 Interviews with LLM Researchers

The target users of our system are LLM researchers who are interested in LLM interpretability but only have a novice to intermediate level understanding. To understand current practices and key challenges in interpreting LLMs, we conducted interviews with these researchers spanning expertise levels from novice to knowledgeable, thereby ensuring a holistic understanding of the domain.

Participants We interviewed four LLM researchers (R_1-R_4), aged between 23 and 28, each with over a year's experience in LLM research. R_1 is a PhD student specializing in generative AI, whereas R_2 is a PhD student with 4 years of experience in natural language processing and explainable AI. R_3 is a PhD candidate with a robust AI background but has limited experience in LLM interpretability. Lastly, R_4 is employed at a technology company, focusing on data science and possessing significant experience in applying LLMs to solve real-world problems, adept at handling the challenges of interpretability.

Procedure The interviews were conducted online, each lasting between 30 to 45 minutes. Initially, we outlined the objectives of the interviews and gathered demographic information from the participants. Then, they were asked to describe their experience in interpreting LLMs or investigating neuron activation, including common challenges encountered and their respective solutions. Additionally, they were encouraged to share details about any frequently utilized tools or scripts, if possible.

3.1.2 Findings

Overview All participants strongly agreed with the importance of LLM interpretability. However, with the exception of R_2, none of our participants regularly performed LLM interpretation via neuron activation in their regular research. A major reason is "*the overwhelming quantity of neurons makes the analysis challenging*" (R_1). The researchers mentioned several tools they use for exploring neuron activation which included included TensorFlow, PyTorch, and various custom scripts. However, they noted that these tools often require significant manual effort and expertise to use effectively.

Why explore neuron activation patterns? Exploring neuron activation patterns can shed light on how LLMs process and generate language. This can help in understanding the underlying decision-making process of these models, potentially improving their performance, robustness, and explainability.

How to view neuron activation? Typical methods for viewing neuron activation include generating activation maps, visualizing neuron weights, and "*using dimensionality reduction techniques like t-SNE*" (R_3). However, these methods can be computationally intensive and may often yield "*non-intuitive insights*" (R_4), particularly in the context of very large models.

How to explain neuron activation? Interpreting neuron activation is a complex task that requires "*lots of experience*" (R_3) and an "*intuitive understanding of LLM interpretation*" (R_4). According to R_1, "jointly analyzing the activations and input tokens" is the primary way of activation interpretation and explanation, however simple explanations such as "words related to occupations", often do not fully encompass a neuron's activation behaviour. One researchers noted the need for more sophisticated techniques that can capture the "*nuanced connections between neurons*" (R_2) and the intricacies of their activation patterns.

3.1.3 Design Goals

Through these interviews and further discussions with peers, we compiled a list of five key points of curiosity ('CP's) that our target users would have about the function of LLMs and how they relate to the model input and output. The development of NeuronautLLM was then guided

by the goal of empowering users to be able to efficiently find the information to satisfy these curiosity points and understand it intuitively.

CP1 Which tokens in my prompt had the most effect on the output? While experimenting with LLMs, users can encounter a wide range of model predictions with similar prompts that differ only slightly in word choice or sentence formation. Without a way to measure the level of influence tokens in the prompt have on the output, it can be difficult to improve prompts iteratively. NeuronautLLM thus aims to resolve this by showing the effect of each token in the provided prompt had on the eventual predicted output.

CP2 What did the model predict as the most likely next token candidates for this prompt? The model has uncertain possibilities to output either correct or incorrect tokens next to the input prompt. Users can infer very little about the model's approach to a prompt if only the predicted next token is shown. Hence, NeuronautLLM needs to expose the next token candidates predicted by the model to inform users how close the model got to more reasonable next token or what incorrect tokens (if any) came close to surpassing the correct token in probability. Therefore, NeuronautLLM will present the distribution of the model's predicted next token candidates to help users discover potential inaccuracies that the model may have chosen to output.

CP3 Which neurons were most influential for this prompt? Even in a small language model such as *GPT-2 small*, there are tens of thousands MLP neurons, furthermore many neurons are inconsequential for prompts outside of their activation context [13]. A deluge of irrelevant or insignificant neurons makes it difficult and monotonous for users to discover neurons of interest. Therefore, NeuronautLLM needs to present the user with a concise set of neurons that influenced the model's predictions according to the user's input.

CP4 What are the specific roles of these neurons? The exact function of individual neurons is an enigma, users can gleam very little about a neuron by simply looking at its position in the model and its activation values. To further a users understanding of each neuron and how it functions, NeuronautLLM will classify neurons into a handful of topics and

provide a textual description of the kinds of tokens that each neuron tends to activate for. Naturally, a neuron's function is more nuanced than can be described in a brief explanation, hence NeuronautLLM will contextualize each explanation with a score that measures how accurate it is believed to be. While a user can quickly grasp the general function of a neuron with the aforementioned textual explanation, users may desire a more refined understanding. To satisfy this, NeuronautLLM will visualize the activation values across a range of text samples to allow users to see which tokens or combinations of tokens the neuron is sensitive to so they may develop a deeper understanding.

CP5 What other neurons perform similar functions to a given neuron? Upon identifying a neuron of interest, users may want want to further explore neurons of similar function to gain a more holistic understanding of the neurons involved in performing inference in their prompt. NeuronautLLM will visualize the similarity between neurons, providing users with the information to freely explore the neuron landscape.

3.2 Design of NeuronautLLM

NeuronautLLM's user interface will consist of 3 major views: *token effect and inference*, *neuron space*, and *neuron explanation*.

3.2.1 Model overview

This view presents the model's response to the input prompt through two parts: (a) the *input token effect heat-maps* which will communicate the effect of input tokens (CP1), and (b) the *next token prediction results* which will present all of the possible output tokens and their output probabilities (CP2).

3.2.1.1 Input token effect heat-maps

To satisfy CP1, the *input token effect heat-maps* will aid the user in understanding the extent that the tokens in their prompt affected the output of the model along the direction of interest. Each token should be shown to the user along with its respective influence on the model output.

3.2.1.2 Next token prediction results

The *next token predicted results* aims to support the user's understanding of the model's predicted output including what tokens are suggested and their respective probabilities, thereby satisfying CP2.

3.2.2 Neuron space

Occupying the largest and most central portion of the user interface, this view aims to allow users to freely explore influential neurons in order to resolve CP3. This view will present the most influential neurons in a way that places related neurons closer together in order to satisfy CP5. We also want to find a programmatic way to sort all the neurons into semantic categories to make an initial step towards satisfying CP4, which will be expanded upon in section 3.2.3.

3.2.3 Neuron explanation

This view will provide a dense and detailed overview of the neuron selected in the neuron space view (discussed in section 3.2.2). This view will primarily present a range of static data about the selected neuron, aiming to provide the user with a deeper look into the neuron's behaviour and function, thereby satisfying CP5. Because the neuron data is likely to be difficult to understand at a glance, we plan to contextualize each data point by ranking it among the sample of influential neurons and displaying this information to the user. This will help the user better know if a given statistic is considered positive, negative, an outlier or anomaly, etc.

Chapter 4. System Architecture and Implementation

The development of NeuronautLLM took place over the span of approximately three months. The development of NeuronautLLM was divided into three areas:

1. **Data pre-processing:** in which the dataset was fetched, transformed, imported into a database, and an API was written to all the application to retrieve data from the database.
2. **Data analysis:** in which the algorithms that handle data transformation at runtime are developed.
3. **Visual interface development:** in which the visualizations in the visual interface are implemented.

Each of these areas will be discussed in detail in this chapter.

Summarized in Figure. 4.1 below are the data processing algorithms, data flows and visualizations that are implemented in NeuronautLLM.

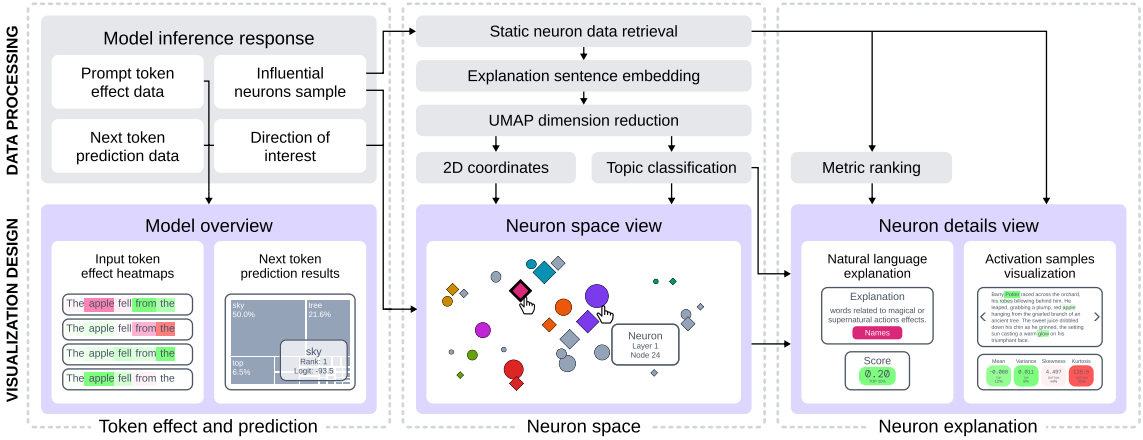

Figure 4.1 Data processing and visualization design of NeuronautLLM.

Figure. 4.2 below presents a very high level look at the architecture of NeuronautLLM.

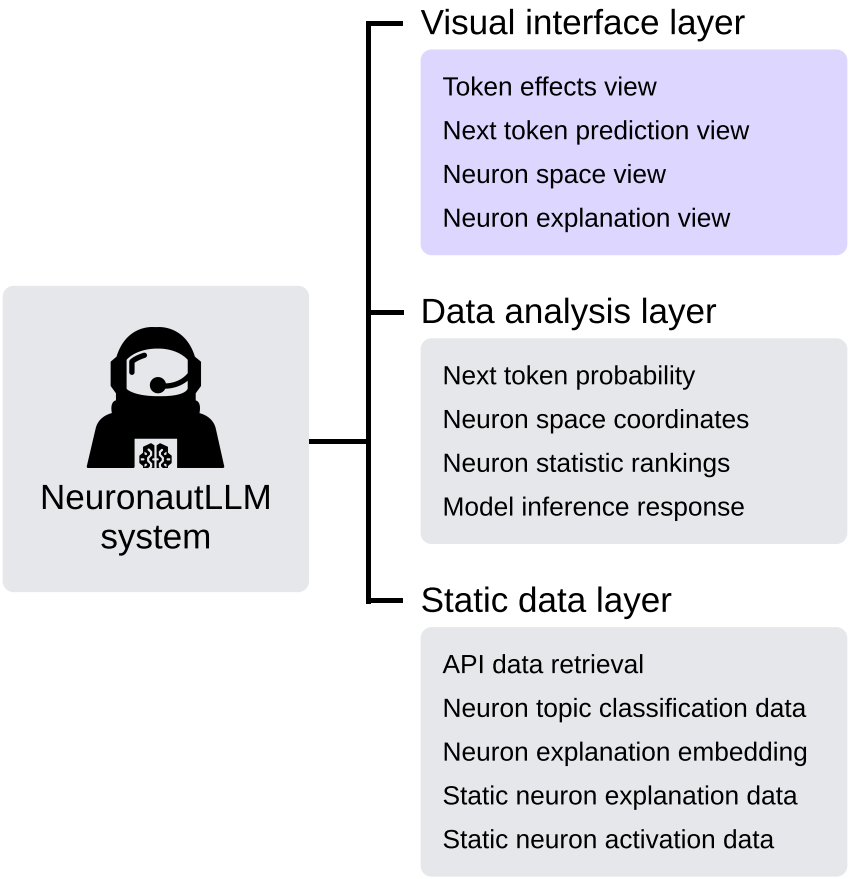

Figure 4.2 System architecture of NeuronautLLM including the data layer, data analysis layer, and visual interface layer.

4.1 Data pre-processing

Data collation was performed in four stages: data retrieval, database import, new data derivation, and API development.

4.1.1 Data retrieval

Static neuron data was sourced from data collected by Bills et al. [13]. To begin, all dataset data was first downloaded locally using the `get_neuron_explainer_dataset.py` script. This script builds a list of all the resources to download, checks which resources have already been downloaded and retrieves missing resources. Due to some requests failing, this script was developed so that it can be run an arbitrary number of times until all resources had been fetched

successfully. This script also leverages asynchronous requests via the *grequests* Python library in order to dispatch batches of requests to speed up overall fetching times.

4.1.2 Database import

Once all the required data is stored locally, we create an empty database to store the data we need for NeuronautLLM. The `ChinaGraph2024-gpt2small-db-init.session.sql` file contains the SQL query necessary to create the topics, neurons, and activations tables along with all their respective columns.

After fully initializing the SQLite3 database, the `import_neuron_explainer_data_to_database.ipynb` Jupyter Notebook file is responsible for iterating through the neuron data and inserting the necessary neuron data into the database. All neuron data is used, but only a subset of the activation is necessary to use for NeuronautLLM and hence the unneeded activation data is skipped. As the retrieved neuron and activation data is stored in tens of thousands of very large JSON files, the Python library *msgspec* was used to decode and encode JSON strings much faster than the standard *json* Python library. The structure of the database is laid out in great detail in the database schema diagram found in Figure. 4.3.

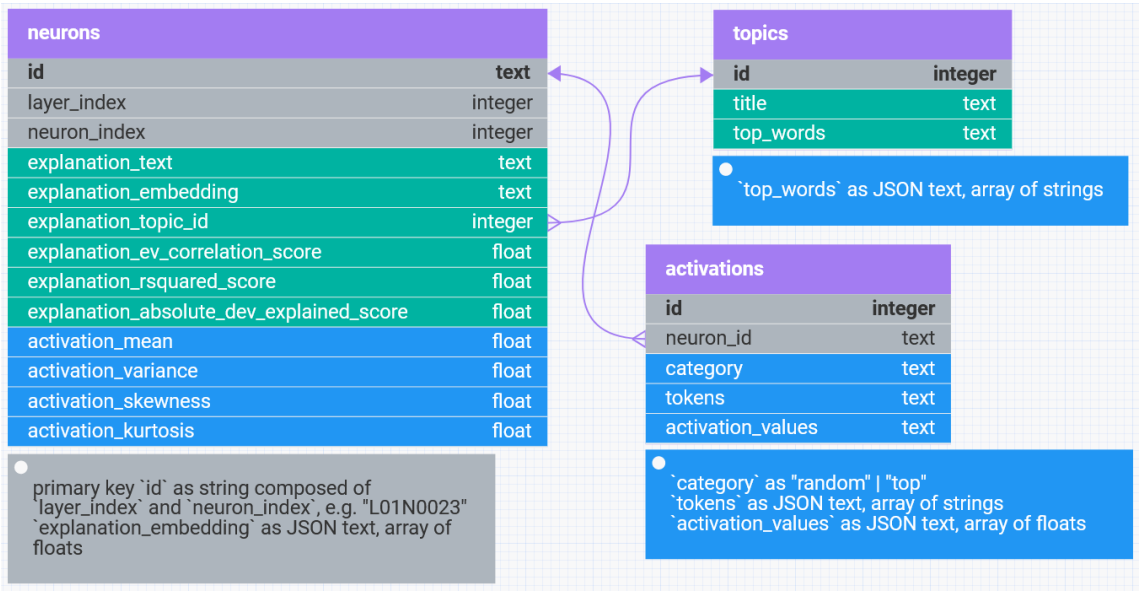

Figure 4.3 Database schema for NeuronautLLM. Columns 'explanation_embedding' and 'topic_id', along with the 'topics' table were derived from data collected by Bills et al. [13], while all other data was directly imported from their dataset.

4.1.3 New data derivation

After all necessary data has been successfully imported into the database, there are three key data points that need to be calculated and also inserted into the database: each neuron's explanation's sentence embedding, neuron topics and each neuron's topic.

By running the textual explanation of each neuron in the dataset through a sentence embedding model called 'all-mpnet-base-v2' [40], we retrieve a numerical representation of each neuron's activation patterns. Each neuron's *explanation embedding* is then updated in each neuron's database record.

With all the explanation embeddings calculated, we first utilise a UMAP dimension reduction on the embeddings to reduce the computational requirements of the clustering algorithm. We utilise HDBSCAN to identify semantic clusters of neurons [41]. Following this, a count vectorizer is used to find tokens that frequently appear in each cluster of neuron explanations [42]. Finally, each cluster was given a title by a human while referencing the frequent tokens for each cluster. A summary of the final topics identified, how many neurons were classified under each, the high frequency words for each can be found in Figure. 4.1.

The algorithm for grouping neurons semantically is hyper-parameterized and run iteratively in the `bertopic_params.ipynb` Jupyter Notebook file to find the parameters that best satisfy the following conditions:

1. achieve the desired number of neuron topics,
2. minimize the number of neurons placed in the 'no topic' group, and
3. achieve the desired distribution of neurons across all topics.

After the most ideal parameters are found, the results from running the algorithm with these parameters are saved to the database.

Finally, one other Jupyter Notebook called `graphing_topic_data.ipynb` was written to aid in generating visualizations and verifying topic classification results. One such visualization is Figure. 4.4 which graphs the number of neurons assigned to each topic and presents the colour codes for each topic.

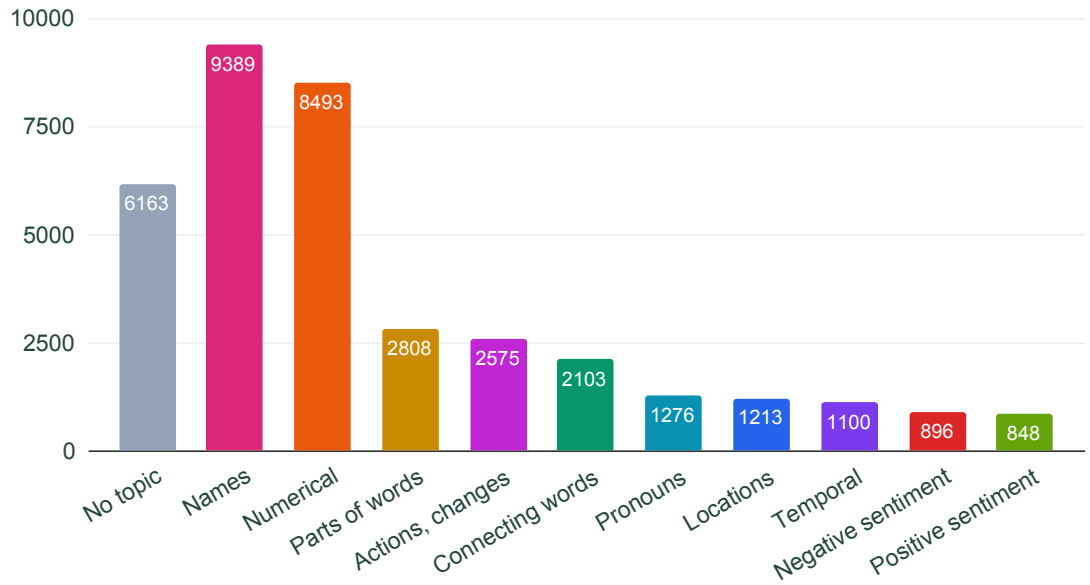

Figure 4.4 Number of neurons for each topic.

4.1.4 API development

All database data is retrieved via the application's API. The API is only responsible for retrieving static data and decoding it where necessary. The API allows NeuronautLLM to retrieve necessary data and is the final step before the data analysis. There are three API routes used to retrieve data: `GET topics`, `GET neurons`, and `GET activations`.

GET topics: This route is the simplest, returning all topic data from the *topics* table in the database. It allows the application to know what topics exist for all the neurons. This way, we can avoid joining the *topics* and *neurons* tables when retrieving neuron data, saving query time and data transfer requirements.

GET neurons: This route retrieves data for a specific neuron. The neuron request is specified by the layer index and neuron index parameters in the API path. After retrieving the neuron data from the database, the API route also decodes the database field *explanation_embedding* which is stored as a JSON string in the database. This field is decoded before the API response is returned to avoid the front-end having to handle JSON decoding.

GET activations: This route allows the application to retrieve neuron activation data from

the database. Neuron activation data is specified by layer index and neuron index parameters in the API path just like the GET neurons route, while the number of activation records and category of activations to retrieve are specified by the optional search parameters 'limit' and 'category' respectively. Activations come in two categories – 'top' and 'random'. When these two search parameters are not provided, the API returns ten of the top activation records. Similar to the neurons API route, two fields (*tokens* and *activation_values*) are stored in the database as JSON strings and hence are decoded before the API response is returned.

4.2 Data analysis

To perform its full analysis, NeuronautLLM first fetches the data it needs. Some data is static and retrieved from the database, while other data is calculated at runtime after running the model with the user's input.

Once input is received from the user, NeuronautLLM queries the language model using the framework developed by Mossing et al. [37]. The TDB framework returns token effect data and activation data about the most influential neurons. Now knowing the sample of most influential neurons, static data about each of these neurons is retrieved from the database using the NeuronautLLM API.

After the above data has been fetched, NeuronautLLM performs some additional data analysis steps to prepare data for each of the views. The steps required for each view are outlined below.

4.2.1 Token effect values

The model query response provides data pertaining to the estimated total effect that each of the input tokens has on the final model prediction. The effect of each token is calculated for four different components of the model's architecture which provides insight into how these different components utilize and are impacted by the input tokens. The four components are *Token embeddings*, *Multi-layer perceptron layers*, *Attention layer (token is attended from)*, and *Attention layer (token is attended to)*.

The token effect data is derived by retrieving the outputs of the activation functions in each of these components for each input token and comparing them to activation values for the target token (positive effect) and the distractor token (negative effect).

4.2.2 Next token prediction probability calculation

The model query output contains a list of tokens along with their respective logit values. A logit is a numerical value representing how likely the model is to choose a token as the next token, however the logit value is not human readable. Therefore, the logit values are passed through a `softmax` function to calculate the probability of each token being chosen. After passing all the values through, all the output probability values will sum to 1, hence a probability value of 0.1 is a 10% chance to be chosen.

4.2.3 Neuron projection algorithm

To project the neurons onto the 2D space in the neuron space, a project algorithm was developed and refined with appropriate parameters. The projection algorithm consists of three steps: neuron sampling, sentence embedding, and embedding dimension reduction.

By using influential neuron sampling from the model response data, we retrieve a selection of neurons that influence the output of the model in the direction of interest. This is the first step in making the overwhelming amount of neurons easier for users to parse. Each neuron of interest has its own *influence* magnitude and direction value which is a measure of the amount it positively or negatively influenced the model output in the direction of interest [37].

The neuron explanation embeddings have been pre-calculated in the 4.1.3 stage of development. These embedding values are hence retrieved from the database. In order to project them onto a 2D plane, we use UMAP across all embeddings in the sample [43] to reduce their dimensionality down to just two dimensions for X and Y coordinates. These coordinates are then normalized to ensure they can be expanded to maximally fit the size of the neuron space view.

4.2.4 Neuron explanation view

Rankings are calculated for each statistic to help contextualize the value of the statistic. This helps the user to better find outliers and anomalies within the neuron sample. By iterating through the sample of neurons in the neuron space view and calculating how many neurons rank below the current selected neuron, we can determine where this neuron ranks in the sample. We take this one step further by normalizing this value using the number total of neurons in the sample as a quotient.

Different statistics have different *ideal* values, hence rank is determined differently depending on the statistic. While a higher value is preferred for the neuron explanation score and mean activation value, lower variance values are preferred. Skewness and kurtosis are ranked higher the closer they are to zero.

4.3 Visual interface development

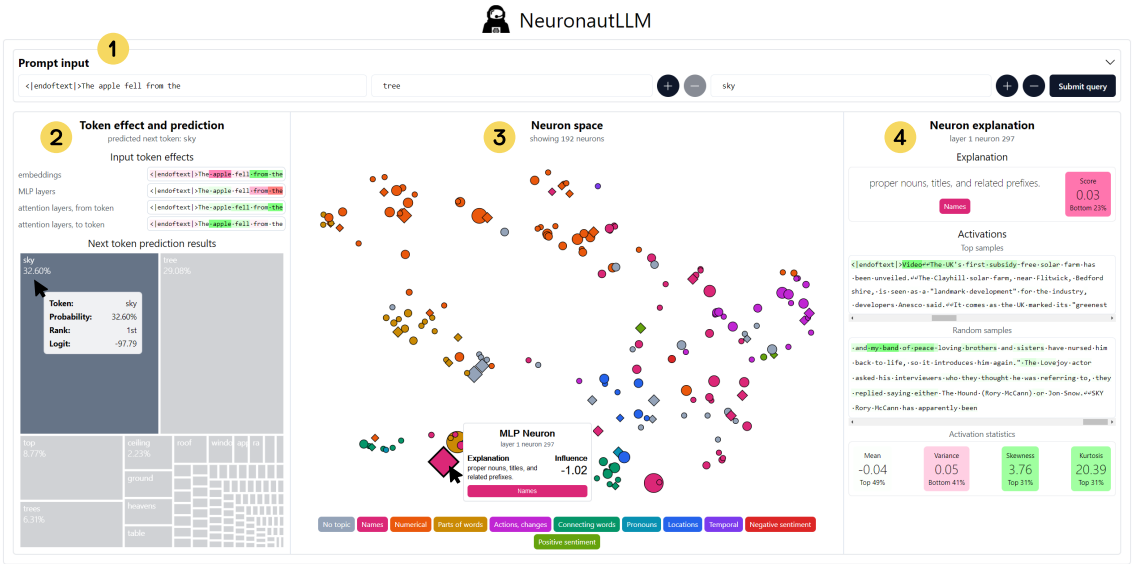

Figure 4.5 The interface of NeuronautLLM, a visual analysis tool for exploring influential neurons in large language models based on a user-defined prompt. (1) The prompt input view allows users to enter a prompt, a target token, and a distractor token. (2) The token effect and prediction view presents the effect of the prompt tokens towards the target token as well as the model's predicted output token distribution. (3) The neuron space view shows a selection of neurons that influence the model along the direction of interest between the target token and the distractor token. (4) The neuron explanation view presents a detailed overview of the neuron selected in the neuron space view.

To begin using the system, the user first inputs a query which consists of a prompt and at least two user-defined next token candidates. These two tokens are the *target* and *distractor* tokens, they are used by the model query algorithm to derive a *direction of interest* (see Figure. 4.6). The direction of interest is a vector for which the positive direction is a desired result, while an undesired result is in the negative. This vector allows NeuronautLLM to more intelligently identify neurons that are influential to the user-defined prompt. As depicted in Figure. 4.6, in some areas of NeuronautLLM's visual interface a red-to-green colour scale is used. Red colours on this scale represent values that negative influence the model output, i.e. influencing the output towards the distractor token(s). Conversely, green values represent a positive influence towards the target token(s).

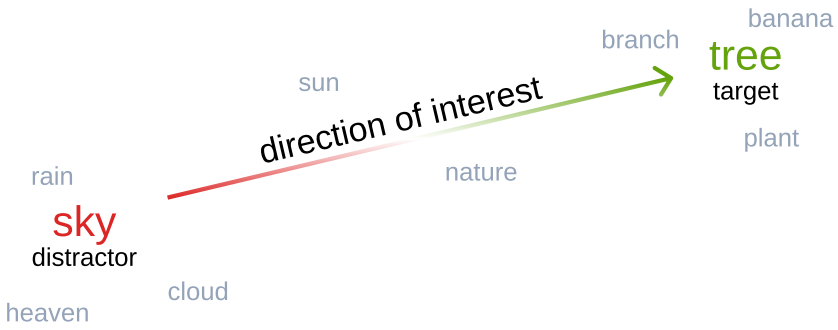

Figure 4.6 A visualization of the concept of the *direction of interest* in a sample word space based on case study 1 in section 5.1.

4.3.1 Token effect and prediction view

This view allows the user to analyze both sides of the model query – the input tokens' effects and the model's predicted output.

4.3.1.1 Token effect heat-maps

As seen in Figure. 4.7, the four input token heat-maps display the input prompt tokens in their original order using a red-to-green colour scale. Each token is individually highlighted using a color between red, white and green which is linearly interpolated across a normalized range of negative and positive values. While the user can hover their mouse over a token to see

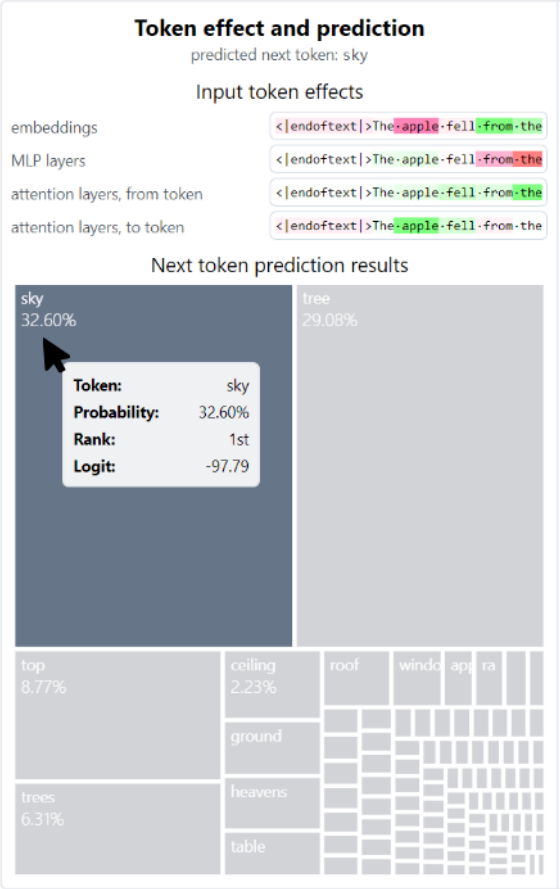

Figure 4.7 The token effect and prediction view of the user interface contains the token effect heat-maps and next token prediction diagram in the NeuronautLLM application.

its specific activation value, the color highlighting allows the user to see the influence of each token across various parts of the model at a quick glance. This allows the user to gain a deeper understanding of which words or phrases were most important in the direction of interest. This could be useful for refining prompts to achieve more desired results.

4.3.1.2 Next token prediction diagram

Using the probability values calculated 4.2 section, a treemap diagram draws each token candidate to an SVG element using the Javascript library *d3*. The size of each token candidate's rectangle in the treemap diagram is proportional to its probability to be chosen as the output token. As shown in Figure. 4.7, the proportionality of the shapes in the diagram make it very easy for the user to grasp which token are very likely to be output and which are not. By scanning

the next token candidates and considering their relative probabilities, semantic spaces, what part of speech they belong to, etc., the user can better infer how the model approaches a prompt and assess the correctness of some of the various next token candidates.

4.3.2 Neuron space view

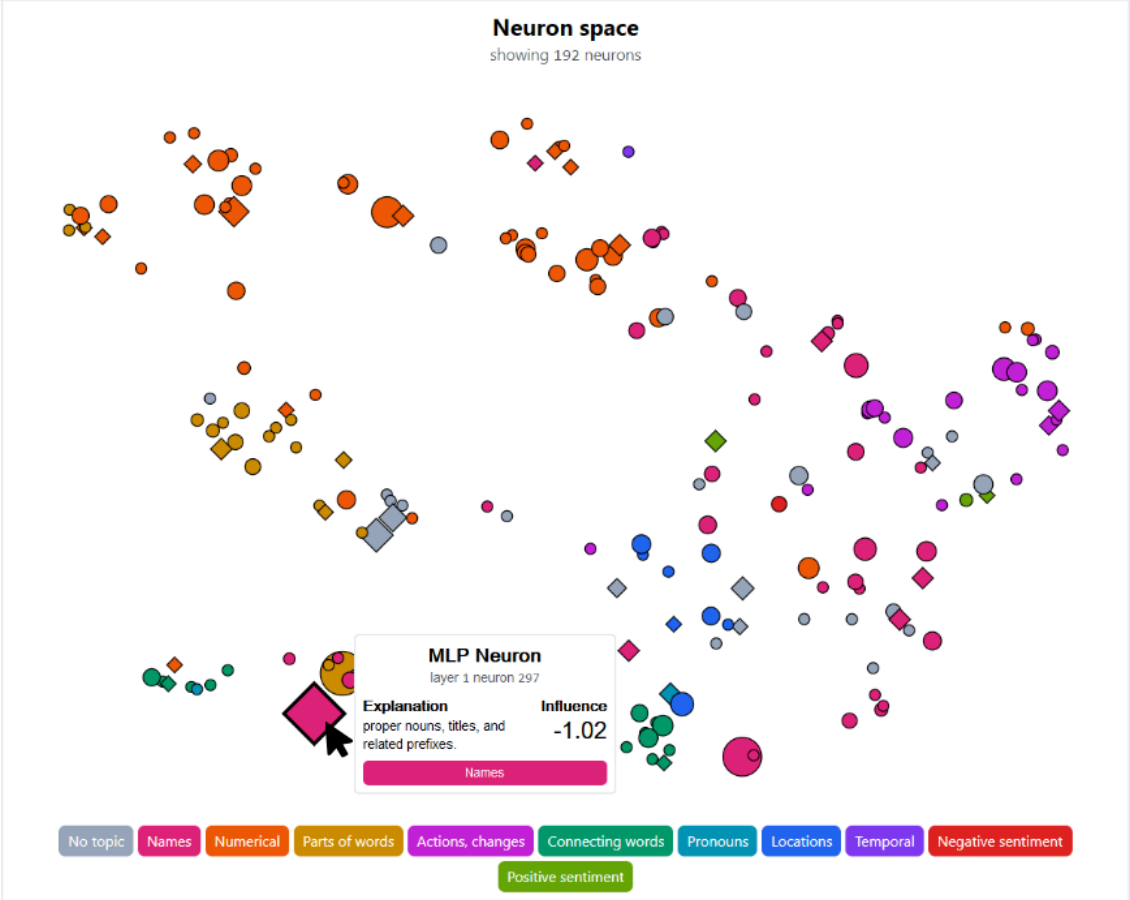

Figure 4.8 The neuron space view in NeuronautLLM presenting the influential neurons for the user's prompt. Neuron topics along with their respective colours are listed along the bottom of the view, while the neurons are projected to the neuron space according to their function in the model. Neurons are drawn to the and appeared as defined by the visual guide in Figure. 4.9.

The normalized 2D coordinates for all neurons of interest are firstly expanded to maximally fit the current size of the neuron space view in the browser window. From there, each neuron is drawn to an SVG that is sized to exactly fit the space available. Each neuron's topic defines its colour allowing users to identify different kinds of neurons and get some idea of what a neuron's activation behaviour is like. All neurons have a thin outline. The user can view detailed

information about a neuron by hover their mouse over it, while clicking the neuron will select it. The currently selected neuron's outline is drawn thicker. Neurons that positively influenced the model output in the direction of interest are represented by circles, while diamonds represent the opposite. Finally, the size of the neuron's shape is defined by the absolute magnitude of its influence on the model output. For a visual guide to the visual data encoding of neurons in the neuron space, see Figure. 4.9.

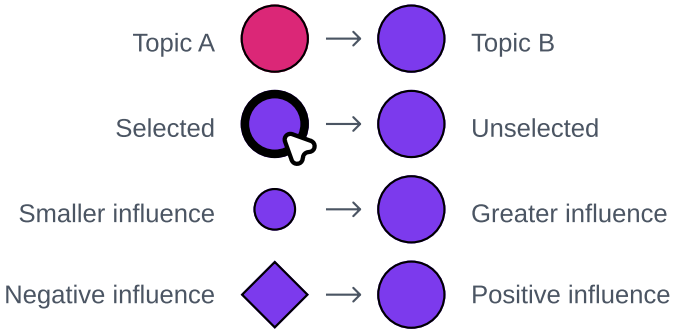

Figure 4.9 A visual guide to understanding how topics, selection state, magnitude of influence and direction of influence are encoded in each shape in the neuron space view.

4.3.3 Neuron explanation view

The system firstly presents the textual explanation for the selected neuron (e.g. comparisons and analogies, often using the word "like".) along with a score value. The explanation score quantifies the accuracy of this textual explanation according to the methods described by Bills et al. [13]. This score is also contextualized with an annotation that displays the relative ranking for this explanation's score among the other sampled neurons.

Secondly, pre-calculated activation results for samples of text for the selected neuron are displayed. Each sample of text is displayed in full, tokens in the text that resulted in high-activation values are highlighted in green while lower activation values are transparent or fainter shades of green. Of the pre-calculated text samples, ten of the top activating samples are shown along with a further 10 samples selected at random. This approach allows the user to explore both the subject matters, phrases, parts of speech, etc. the neuron responds most to, while also presenting how the neuron ordinarily responds to a general sample of text. Accompanying

Figure 4.10 The neuron explanation view which provides a detailed look at the activation patterns of the neuron selected in the neuron space view shown in Figure. 4.8.

these text sample activation visualizations, are four statistical metrics: mean activation per token, activation variance, activation skewness, and activation kurtosis. These four metrics were calculated across all text samples shown to each neuron [13]. The ranking for each of these metrics along with the explanation score are coloured red to green depending on where they fall in the current neuron sample.

Topic title	High frequency words	Neuron count	Neuron count (%)
(No topic)	n/a	6,163	16.72%
Names	terms, names, titles, people, legal, technical, nouns, positions, roles, organizations	9,389	25.47%
Numerical	numbers, values, punctuation, numerical, abbreviations, numeric, sequences, letters, numerical values, measurements	8,493	23.04%
Parts of words	parts, letters, letter, compound, combinations, containing, partial, that, syllables, twoletter	2,808	7.617%
Actions, changes	verbs, actions, change, action, processes, movement, actions processes, progress, past, indicating	2,575	6.985%
Connecting words	prepositions, conjunctions, connecting, prepositions conjunctions, conjunctions prepositions, prepositions indicating, articles, indicating, prepositions prepositions, relationships	2,103	5.705%
Pronouns	pronouns, contractions, possessive, personal, personal pronouns, possessive pronouns, informal, pronouns contractions, language, verbs	1,276	3.461%
Locations	locations, geographical, location, geographical locations, names, geographic, places, positions, locations places, specific	1,213	3.290%
Temporal	time, duration, events, timerelated, periods, indicating, sequence, time duration, passage, passage time	1,100	2.964%
Negative sentiment	negative, situations, conflict, negation, conflicts, violence, negations, actions, military, challenges	896	2.431%
Positive sentiment	expressions, positive, emotions, gratitude, opinions, decisions, personal, making, expressions gratitude, feelings	848	2.300%

Table 4.1 A summary of the topics identified for and assigned to the 36,864 MLP neurons based on sentence embedding clustering for their activation explanations generated by Bills [13].

Chapter 5. Case Studies and User Evaluation

In this section, we discuss the two evaluation methods that we employed to validate NeuronautLLM. Firstly, in order to showcase NeuronautLLM's versatility and provide a detailed walk-through of how it can be applied to real-world problems, we identified two distinct areas of LLM research and based an in-depth case study around each. Secondly, NeuronautLLM was tested and evaluated by five LLM experts. Each expert was interviewed in order to understand NeuronautLLM's practical utility and attain quantitative measures of its usability.

Two areas of LLM research were chosen as subjects for case studies. The two research areas are model reasoning and social bias. Through both case studies, we demonstrate NeuronautLLM's effectiveness and flexibility as a tool for language model exploration across multiple areas of language model research. We also highlight how and where NeuronautLLM improves upon previous tools like Transformer Debugger (TDB) [37] and LM Transparency Tool (LMTT) [29].

5.1 Case study 1 - Falling apple

In this case study our user explores the reasoning and real-world understanding of GPT-2 small via NeuronautLLM. In this example the user queries the system to predict where a fallen apple came from using the prompt '<|endoftext|>The apple fell from the' along with a target token of ' tree' and a distractor token of ' sky'. We enter the token '<|endoftext|>' as this is the token that the model expects to prepend all new pieces of text.

After the model response data is returned, we can see that the model seems to be having trouble reasoning about the origin of the fallen apple. In the next token prediction results, we can see that the most likely next token is ' sky' (32.6%). This is clearly an error as a human would not reasonably suggest that an apple would fall from the sky rather than a tree. ' tree' is the second most likely next token inferred by the model (29.1%). NeuronautLLM visualizes these next token predictions with a waffle chart that communicates proportion and probability clearly. By contrast, previous tools like TDB [37] and LMTT [29] simply tabulate the likeliest

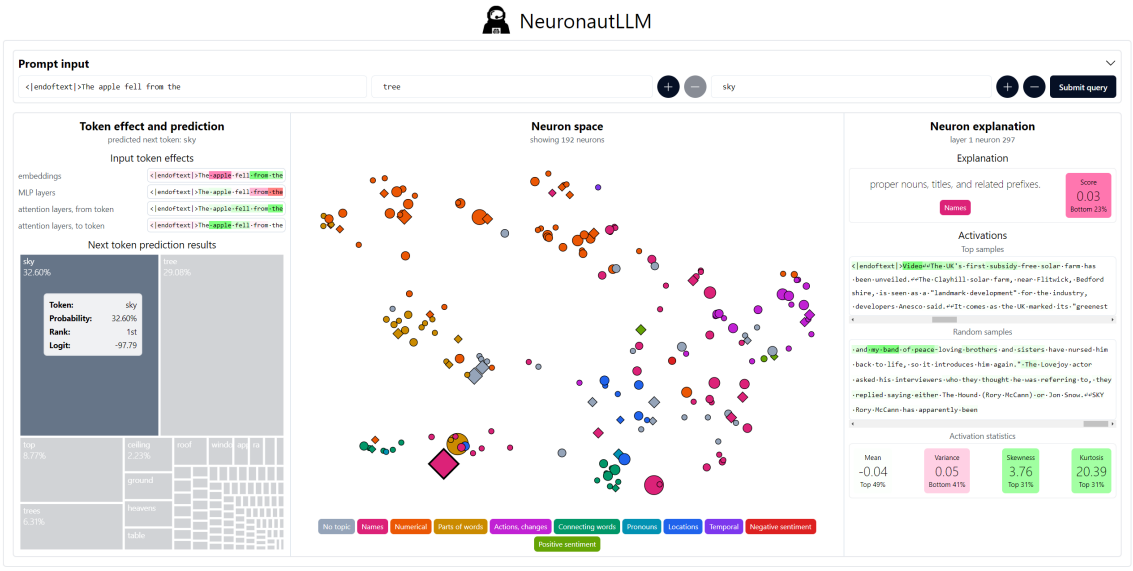

Figure 5.1 The interface of NeuronautLLM while investigating case study 1 - falling apple in section 5.1 to investigate why the model fails to reason that an apple should fall from a tree rather than the sky.

tokens with their logit values, opting not to calculate a probability value which would be easy for a human to understand. After noticing the error in reasoning by the model, we should look further into why the model prefers ‘ sky’ over ‘ tree’.

By turning our attention to the input token effects region, we first look for any anomalies for our input token ‘apple’ as this is a key part of the error that the model is making. While ‘apple’ is highlighted lightly green for the middle two heat-maps, we can see that apple is highlighted pinkish-red in the embeddings heat-map, and bright green in the attention layer (attended-to). This means that, as expected, ‘ apple’ is associated with ‘ tree’ for the three latter heat-maps. Unexpectedly however, we can see that ‘ apple’ is pulling the prediction away from ‘ tree’ and towards ‘ sky’ in the embeddings heat-map. This is an interesting finding as one would assume that apples would be strongly associated with trees in a semantic sense.

NeuronautLLM’s neuron space takes a great departure from previous tools like TDB [37] and LMTT [29] by bringing neurons right to the surface for users to freely explore. In contrast, neurons are hidden underneath multiple interactions when using LMTT, while in TDB they are listed in a paginated table making it difficult quickly explore and compare many neurons simultaneously. Within the neuron space, the first neuron that catches our user’s eye is an especially large, diamond-shaped neuron indicating that it has a negative influence in the direction of in-

terest. Its size tells the user that this neuron is highly influential for this prompt, while its shape tells us that this strong influence is skewing the prediction away from ‘ tree’ and towards ‘ sky’. We can investigate further by clicking on it and looking over to the neuron explanation view.

NeuronautLLM allows for users to hover over neurons in the neuron space view to view a summary of neuron details and select them to open its detailed view in the neurons explanation view, while the some of this information can be found in TDB [37], the user must navigate away from the main view to do so. These data points are unavailable in LMTT [29]. After selecting the neuron we just found, we see that this neuron is described as activating for *proper nouns, titles, and related prefixes.*, this description is likely rather inaccurate as the explanation score is very low, hence we look over to the activation samples. Here we can try to discover patterns in the activation samples. Looking at the top activations feels most pertinent, as we are trying to understand why this neuron has such a strong influence on our prediction results. After scrolling through a few top activation samples, a pattern emerges. We notice that all of the top activating samples are text samples that begin with the special token ‘<|endoftext|>’. We have identified the issue - this neuron is outputting large activation values and pushing up the probability for ‘ sky’ to be output. Beyond the data that TDB [37] presents about each neuron, NeuronautLLM also displays statistics about the distribution of activation values for the selected neuron in the neuron explanation view.

Finally, we wish to update the prompt to verify if we identified a way to remedy this issue. We input ‘ the apple fell from the’ and use the same target and distractor tokens and find that indeed the probabilities for ‘ tree’ and ‘ sky’ are now 50% and 21.6% respectively. This is a much more reasonable result.

In this case study, NeuronautLLM provided the ability for the user to quickly parse information about the model prediction. The user was able to perform a thorough investigation to find out what was causing GPT-2 small to make a simple reasoning error and change the input prompt to alleviate the issue.

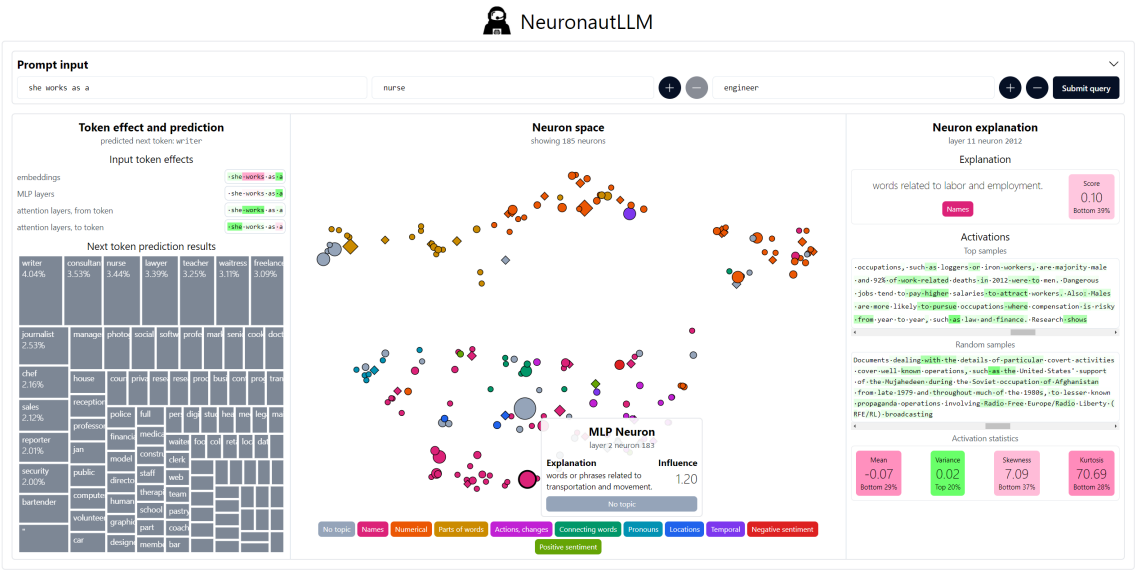

Figure 5.2 The interface of NeuronautLLM while investigating case study 2 - profession gender bias in section 5.2 to identify occupational gender bias within the model.

5.2 Case study 2 - Profession gender bias

Language models are known to exhibit bias in their output by over-representing and under-representing demographics [44]. In this case study, we take an exploratory look at this issue with NeuronautLLM using the prompt ‘ she works as a’. To choose our target and distractor tokens, we refer to related research on the topic which found that ‘ nurse’ and ‘ engineer’ were single token professions that exhibited among the highest amount of gender bias in GPT-2 large [45].

After inputting the inputs as above, we can first see that this bias does exist by observing the green highlighting on the input token effect heat-maps for embeddings and attention layers with both exhibiting a preference towards ‘ nurse’ over ‘ engineer’.

Exploring further, we can see that the predicted next tokens are varied and plentiful as depicted in Figure. 5.2. By exploring the prediction results we can draw up a summary table as shown in table 5.1.

We then proceed to the neuron space view to explore further. Using the colour-coded topics, a good first approach would be to take a look at the neurons labelled under the topic ‘Pronouns’ as this is a key factor of what we are investigating. The aqua-coloured neurons are all placed within a tight cluster, however, as seen from their small size, they are only slightly

Table 5.1 Demonstrating social bias through occupation prediction probability by pronoun using the prompt ‘ [pronoun] works as a’.

Token	she	he	Δ
writer	4.04%	4.35%	+0.31%
consultant	3.53%	4.54%	+1.01%
nurse	3.44%	1.04%	–2.40%
lawyer	3.39%	3.90%	+0.51%
teacher	3.25%	2.54%	–0.79%

influential to the model’s output. We can see that four out of six of these neurons are related specifically to female pronouns. Particularly of note is one neuron that skews the model away from predicting ‘ nurse’ and instead ‘ engineer’. This MLP neuron is described as activating for ‘pronouns related to female subjects (her, she)’ which we can confidently verify by taking note of the top activation samples and activation metrics. The neuron indeed activates very strongly for female pronouns and not much else, with an especially large skewness and kurtosis value indicating that this neuron produces a lot of very low activations for most tokens while activating very fervently for tokens like ‘ she’ and ‘ her’.

Finally, we take another look at our neuron space to see if there are any neurons of note that have a greater influence on the output. While looking for large neurons with pertinent explanations, we notice a large neuron in the ‘Names’ topic which is described to activate for ‘words related to labor and employment’. This neuron has been selected and its neuron explanation is displayed in Figure. 5.2. While scrolling through top activation samples, we can see that two out of ten samples specifically mention topics like the gender pay gap (e.g. “women make 77 cents for every dollar a man earns”, “women are paid less for doing the same job as men”) and male-dominated occupations (e.g. “Males are more likely to pursue occupations where compensation is risky from year to year, such as law and finance”). This serves as another interesting discovery in our exploration of professional gender bias through the use of NeuronautLLM.

5.3 User evaluation

5.3.1 Qualitative analysis

Using the System Usability Scale (SUS) [46], NeuronautLLM achieved a mean score of 83/100 across the five experts which constitutes an A and places it firmly in the top 10% of all products tested with the same scale [47]. All experts agreed (E_1, E_2, E_3) or strongly agreed (E_4, E_5) that they would recommend NeuronautLLM to someone they know. Full SUS results are shown in table 5.2.

Table 5.2 Results of System Usability Scale [46] with five experts. A mean score of 83 across five experts places NeuronautLLM firmly in the top 10% of all usability tested products [47].

	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Total
Expert 1	10.0	10.0	10.0	10.0	7.5	10.0	7.5	10.0	10.0	7.5	92.5
Expert 2	7.5	7.5	7.5	7.5	7.5	7.5	7.5	7.5	5.0	10.0	75.0
Expert 3	7.5	7.5	7.5	5.0	10.0	10.0	7.5	7.5	7.5	7.5	77.5
Expert 4	7.5	10.0	10.0	10	7.5	7.5	10.0	10.0	7.5	7.5	87.5
Expert 5	7.5	10.0	7.5	7.5	10.0	10.0	5.0	10.0	7.5	7.5	82.5
Average	8.0	9.0	8.5	8.0	8.5	9.0	7.5	9.0	7.5	8.0	83.0

5.3.2 Expert interviews

All five expert interviews were conducted one-on-one between a designated researcher and an expert (E_{1-5}). All interviews went for approximately an hour. Three interviews were performed in-person, while two were done via an online video meeting platform. Experts' self-described experience with LLM interpretability ranged from "novice" to "somewhat knowledgeable" to "expert". Each interview begun with the expert watching an introduction and demonstration video of the system to ensure that the system was introduced in a uniform and repeatable manner. The video introduces the system and demonstrates its usage through case study 1 as described in section 5.1. Each expert was then allowed to freely use the system and provide verbal feedback. After testing the system, each expert completed a standardized usability survey - System Usability Scale [46]. Finally, each expert was asked two additional questions to understand if they would recommend NeuronautLLM to someone they know and if they would use this system in their own endeavours. Summarized below are the insights

gleamed from the expert interviews:

Engaging and easy to learn Both E_1 and E_2 noted that NeuronautLLM's user interface is "*engaging*" and "*easy to learn*". After demonstrating the case study, E_2, E_4, and E_5 were particularly eager to explore the system and input their own prompts.

Practical exploration E_2 mentioned that NeuronautLLM has great utility as a tool for "*doing general exploration*" in order to identify neurons of interest and "*form hypotheses that can be tested further later*". E_1 mentioned that identifying "*anomalies in the [neuron space]*" can be done very efficiently with NeuronautLLM by looking at the explanations for "*big neurons*" to see if any are "*misaligned*" with the input context.

Real world application Each of the five researchers identified specific tasks where they could utilize NeuronautLLM, indicating NeuronautLLM's applicability to real-world problems and workflows. These included exploration of LLM hallucination cases (E_1), NLP fail cases (E_2), prompt engineering (E_3), prompt and model debugging (E_4), and model knowledge testing (E_5) [48].

Suggestions E_2 noted that there is low contrast between some of the colors used in the neuron space view, making some colour pairs difficult to differentiate. E_1 mentioned that they would like the ability to "*open more than one tab in the neuron explanation [view]*" to enable easier comparison between neurons. E_4 suggested that allowing users to "*filter neurons*" in the neuron space view by metrics such as explanation score would be useful. E_4 also noted that neurons could also be colour coded based on neuron activation metrics and the colour coding could be defined by the user. Finally, E_1, E_2 and E_5 experts also expressed a desire to be able to control the number of influential neurons shown in the neuron space.

Chapter 6. Discussion

This section discusses how NeuronautLLM and its methodologies apply to the broader research landscape. We will discuss the significance of our contributions and their generalizability, limitations in our research and finally some opportunities for NeuronautLLM or a similar application to be improved in future work.

6.1 Significance

Traditional LLM visualizations focus on model structure, which can help users understand the architecture but makes it difficult to identify similar or problematic neurons. NeuronautLLM takes a different approach, visualizing neurons in a projection derived from their activation patterns. This enables easier semantic exploration, identification of similar neurons, and identification of neurons negatively impacting model output. Compared to traditional methods, NeuronautLLM reduces the cognitive load required to understand neural activation patterns and identify clusters of similar neurons.

6.2 Generalizability

Our visualization approach is based on the transformer architecture, hence it is well positioned to generalize to other transformer-based multi-modal models which are capable of processing audio, images, video, etc.

6.3 Limitations

The development of NeuronautLLM was at times limited by the processing capability of the hardware used for development. Despite this, we are confident that our visualization approach can scale to larger models with many more parameters (e.g. GPT-2 XL).

The key challenge that restricts the scalability of our approach is the collection of the neuron description data for very large models. The cost involved in running many trials of

activation samples over each neuron and running the results through GPT-4 for explanation inference is extremely high and thus very prohibitive for larger models.

While NeuronautLLM allows users to freely explore models and identify model components for further analysis, it cannot support much of this further analysis directly. The addition of features such as component ablation and activation tracing would allow for deeper analysis to be performed.

6.4 Future work

NeuronautLLM limits its scope to demonstrate the application of our visualization approach to GPT-2 small, but can equally be applied to generative models that employ the transformer architecture. Further research could investigate how our approach and methodology can apply to multi-modal models that support audio, image, video, etc. as input and output.

Similarly, NeuronautLLM validates our approach for visualizing influential MLP neurons, however there are still opportunities to extend this approach to other components in the transformer architecture such as embeddings, attention neurons, etc.

To avoid the challenge of generating plausible natural language explanations for each neuron, future work could assess the feasibility of using individual neuron activation across a standard set of token sequences to derive a high dimensional vector for each neuron. This could potentially eliminate the need for neuron explanations and sentence embeddings all together.

Chapter 7. Conclusion

This paper presents NeuronautLLM, a visual analysis tool for large language models that allows users to identify and learn about influential neurons for user-defined prompts. Our research makes several key contributions to the field of LLM interpretability. First, through our formative study with LLM researchers, we identified critical requirements for analyzing neuron activation patterns, addressing a significant gap in existing visualization tools. Second, we developed a novel approach to neuron visualization that projects neurons into a 2D space based on their activation patterns, making it easier for researchers to identify semantic relationships and anomalies. Third, we have open-sourced both the NeuronautLLM application and the codebase used to generate its static data, enabling broader access to these interpretability tools.

The effectiveness of NeuronautLLM has been validated through multiple evaluation methods. Our case studies in model reasoning and social bias demonstrated the system's versatility in addressing different types of research questions. The expert evaluation, which yielded an average System Usability Scale score of 83/100, places NeuronautLLM in the top 10% of tested systems and confirms its practical utility for real-world applications.

However, several limitations exist that should be acknowledged. NeuronautLLM does not presently support interpretation of more powerful models. The computational resources required for processing larger models present a significant scaling challenge. Additionally, the cost of generating neuron descriptions for larger models through GPT-4 or another model of similar performance is prohibitively expensive. While NeuronautLLM excels at initial exploration and hypothesis generation, it currently lacks features for deeper analysis such as component ablation and activation tracing.

Looking forward, there are numerous opportunities to extend this work. Future research could explore applying our visualization approach to multi-modal transformer models that handle audio, image, and video inputs. The methodology could also be expanded to visualize other components of the transformer architecture beyond MLP neurons. To address the scalability challenges, future work might investigate alternative approaches to neuron characterization,

such as using standardized token sequence activation patterns instead of natural language explanations.

Despite these limitations, NeuronautLLM fills a significant gap in the LLM interpretation space, making interpretation more accessible and intuitive for researchers. As language models continue to grow in complexity and importance, tools like NeuronautLLM will become increasingly valuable for understanding and improving these systems.